

Developing a Fast Winning Strategy for Toss or Hold'em Pokerbots

Overview of Toss or Hold'em Variant

Toss or Hold'em is a heads-up No-Limit Texas Hold'em variant used in the IAP 2026 Pokerbots competition

¹ . The key differences from standard Hold'em are:

- **Three Hole Cards & Discard:** Each player is initially dealt 3 private hole cards. After the **flop**, which in this variant consists of only **2 community cards** ² ³ , a special **discard round** occurs. The out-of-position player (the big blind) must discard one of their 3 hole cards first, followed by the in-position player (dealer) discarding one ² . The discarded cards are **moved to the board as additional community cards** and are revealed to both players ⁴ ⁵ . After both players discard, there will be **4 board cards** on the table (the original 2 flop cards plus 2 discarded cards) ³ . Play then continues with the turn and river being dealt as normal (adding a 5th and 6th board card) ⁶ ⁷ . At showdown, each player makes the best 5-card hand from their 2 remaining hole cards plus the 6 board cards (8 cards total) ⁸ .
- **Information Asymmetry in Discard:** Because the discards are sequential, the second player to discard gains a positional advantage. The first discarder's card is revealed on the board **before** the second player chooses their discard ⁵ . This means the dealer (in-position on the flop) gets to see what card the opponent threw and how it affects the board before deciding which of their own cards to toss ⁹ . All discarded cards become **public information** and can be used by either player to form a hand (like any board card) ¹⁰ ⁵ . Notably, when you discard a card, **you don't entirely lose its value** – it still can contribute to your hand at showdown as part of the community board, but your opponent also gains access to it ¹¹ .
- **Gameplay Structure:** The betting structure and rounds follow Texas Hold'em, with an extra discard step inserted after the flop dealing ¹² ¹³ . Each hand (round) starts with fresh stacks (e.g. 400 chips each) and standard blinds (1/2) ¹⁴ ¹⁵ . Preflop betting is with 3 hole cards. After the flop is dealt (2 cards), no betting occurs until after the discards are made ² . Then a betting round happens on the "post-discard flop" (4 board cards), followed by turn and river deals with betting rounds as usual ¹³ ⁷ .

This variant's extra hole card and discard mechanic introduce new strategic considerations. Players effectively have the chance to **improve their hand by discarding the weakest card**, but must consider the information and value that discarded card gives to the opponent. The goal is to leverage the available **tools** – fast hand evaluation and Monte Carlo equity simulation – to make optimal decisions at each stage under these rules.

Fast Hand Evaluation Tools (PokerStove & Monte Carlo Simulation)

To win the competition, a **fast and accurate** hand evaluation is critical. The baseline implementation already integrates a two-pronged evaluation system:

- **PokerStove Hand Evaluator:** PokerStove is a high-speed C++ poker hand evaluation library. In this bot, a Python wrapper (via SWIG) is used to evaluate 7-card hands and even “best-of-8” scenarios efficiently ¹⁶ ¹⁷. If PokerStove is available, the bot uses it as the backend for ranking hands; otherwise it falls back to a Python-based library (“Deuces”) for evaluation ¹⁷ ¹⁸. This ensures that computing the strength of a given 5-card hand (or the best 5-card hand out of 7 or 8 cards) is as fast as possible. For example, the Deuces library can evaluate ~15k 7-card hands per second in Python, whereas a C++ evaluator can reach on the order of 140k+ evaluations per second ¹⁹. By using PokerStove’s `CardSet.evaluateHigh()` under the hood, the bot can get exact hand rankings quickly ²⁰ ²¹.
- **Monte Carlo Equity Simulation:** Exact calculation of win probability (equity) against an unknown opponent hand and future board cards can be intractable to do analytically for every scenario. Instead, the bot uses **Monte Carlo simulation** to estimate hand equities. Monte Carlo simulation in this context means the bot will “**play out**” the rest of the hand many times by randomly dealing the opponent’s cards and the remaining community cards, and count how often the hero wins, loses, or ties ²² ²³. By aggregating results of many random trials, it approximates the win probability (equity) of the hero’s hand ²⁴. This is a common approach in poker tools and calculators ²⁵. As one description puts it, the engine effectively “*deals random cards for your opponents and the remaining community cards, then checks who wins... By aggregating the results of all these random trials, it arrives at a highly accurate estimate of how often your hand comes out on top*” ²⁶.
- **Caching and Speed Optimizations:** The baseline code employs heavy caching and time-budgeting to keep simulations fast. Results of hand evaluations are memoized with LRU caches so that the same combination of cards isn’t recomputed repeatedly ²¹. For example, once the best 5-card hand of any 8-card combination is evaluated, it’s stored for quick reuse ²⁷. Similarly, computed equity outcomes for specific states can be cached. The Monte Carlo routines also allow specifying a maximum number of samples or a time limit, so the bot doesn’t overshoot its per-turn time budget ²⁸ ²⁹. By default, the simulation will run only a limited number of samples (e.g. 100-120 trials for equity estimation on later streets) or for a few dozens of milliseconds per decision ³⁰ ³¹. If an exact calculation is feasible (for instance, if very few unknown cards remain), the code can even **enumerate all possibilities** instead of sampling ³² ³³ – this guarantees accuracy in endgame scenarios. The Monte Carlo function also uses deterministic seeding when a fixed number of samples is requested with no time limit, ensuring reproducible results for identical game states (important for consistency) ³⁴ ³⁵.

Using these tools: The strategy will rely on Monte Carlo equity estimates at decision points (like whether to fold, call, or raise) and for choosing which card to discard. The PokerStove evaluator is used internally to rank hands during these simulations and at showdown. Because speed is crucial, the strategy should minimize expensive calculations when possible (e.g., use heuristics preflop or caching), and use the fastest available evaluator for any needed hand ranking. By combining fast C++ evaluation with smart Monte Carlo sampling limits, the bot can make near-real-time decisions with reasonable accuracy on win probabilities

²² ³⁶.

Pre-Flop Strategy with Three Hole Cards

Before the flop, each player has 3 hole cards (one more than normal Texas Hold'em). This means the range of possible starting holdings is wider and the ability to **selectively discard later** will effectively allow a player to keep the best 2 out of those 3. As a result, nearly every 3-card hand has some potential – the worst card can often be thrown away. However, some 3-card combinations are clearly stronger than others (high pairs, suited trips, connected high cards, etc.), and a solid preflop strategy is needed to decide when to enter the pot or when to play aggressively versus fold.

Baseline Preflop Evaluation: The bot uses a heuristic function `preflop_strength()` to assign a value between 0 and 1 to any 3-card starting hand ³⁷ ³⁸. This function is a fast approximate evaluator that considers: - The ranks of the cards (higher is better): it normalizes the sum of ranks to a base value ³⁹. - Pair or trips: having a pair or three-of-a-kind in the 3 cards gives a bonus (e.g. ~0.25 added for trips, ~0.12 for one pair) ⁴⁰. - Suitedness: having 2 or 3 cards of the same suit yields a bonus (3 suited cards ~0.08, a single suit match ~0.04) ⁴¹, since this gives flush potential. - Connectivity: how close the ranks are – small gaps mean straight potential (e.g. if the span of the 3 ranks is ≤ 4 , a bonus ~0.05; if any two are within 2 ranks, smaller bonus) ⁴². - High card presence: additional tiny boosts for each card that is Jack or above ⁴³, reflecting high-card strength.

The result is a quick-and-dirty strength score. For example, a hand like (A♠, A♥, 5♦) would score very high due to a pair of Aces and high cards; (7♦, 8♦, 9♣) would score well for being connected and suited; whereas a random unsuited spread like (2♣, 7♦, 10♠) would score much lower. This **preflop strength metric** is used instead of expensive Monte Carlo simulation at the very start of the hand ⁴⁴, saving time for later decisions. It's critical to start fast because enumerating millions of possible outcomes for 3 vs 3 cards preflop is unnecessary – a heuristic or lookup can suffice.

Preflop Decision Rules: Using the strength metric, the strategy can decide: - **Open-Raising vs. Folding:** If our 3-card hand is strong enough, we should raise preflop to build the pot or force weaker hands to fold. In the baseline, a raise threshold is set by comparing the hand's strength (as an equity proxy) to the pot odds of a raise. For instance, with blinds 1/2, the dealer (small blind) acts first and needs to call 1 more chip into a 4-chip pot (after blinds) – roughly 25% equity needed to call. A hand significantly stronger than that (say >45-50% equity vs random) might merit a raise ⁴⁵. Conversely, a very weak hand (strength far below the cost threshold) can be folded to avoid playing a poor hand out of position. The baseline uses a **margin-based criterion**: for preflop (street 0), it requires roughly an **equity > pot_odds + 0.20** (i.e. at least ~20% above breakeven) to raise, and would fold if equity is ~5% or more below the breakeven threshold ⁴⁶ ⁴⁷. In practice, this means only fairly strong 3-card holdings are raised, marginal ones are just called, and truly terrible ones are folded. For example, if our `preflop_strength` is 0.50 (~50%), pot odds to call are ~0.25, so we exceed by 0.25 which is above the 0.20 raise margin – a raise is justified. - **Calling vs. Folding when facing a raise:** If the opponent raises preflop, we compute our pot odds to call the raise and compare to our hand strength. Since each hand is reset and stacks are equal, the bot can afford to fold the worst hands and wait for a better spot (because there are 1000 rounds total ¹⁴). Generally, if our estimated equity is below the pot odds of calling, we should fold to avoid -EV calls ⁴⁸. The baseline logic explicitly folds if `equity < pot_odds` in a decision and folding is available ⁴⁸. However, a slight **call margin** is allowed where the bot might still call with equity just a bit under the pot odds to avoid being overly tight (this margin is small preflop, ~0.05 or 5%) ⁴⁷. In other words, if our hand is only slightly behind the immediate odds, we may still continue, recognizing there's future play and the discard option to improve. - **No heavy computation preflop:** The strategy avoids Monte Carlo at this stage for speed. Using the heuristic and

some pre-set thresholds is sufficient and much faster. This also makes the bot's preflop actions **deterministic** and easily tunable (we can adjust the raise threshold or folding threshold as needed).

In summary, **play more hands** than you would in normal Hold'em (because with 3 cards you can afford to see a flop and then discard the worst card), **but still fold the truly junk combinations** that have low potential even after a discard. By raising strong 3-card hands, the bot pressures the opponent and possibly steals blinds with premium holdings. By folding the bottom tier, it avoids wasting chips on hopeless cases. The balance between these is set by that strength heuristic and the pot odds comparison.

Example: If the bot is dealt (A♥, K♣, 7♦) as dealer (SB): - It evaluates a high strength (two big cards). Pot odds to call = 0.25 (1 chip to call into 4). Our hand strength might be ~0.65 by the heuristic. This is well above even a raise threshold (~0.25+0.20=0.45). So the bot should come in for a raise. If the opponent re-raises, we reassess – likely still continuing given a strong hand. If the bot is dealt (4♦, 9♠, 10♣): - Middle cards, no suit, weak connectivity (gap 4-9-10). Strength might be ~0.30. If facing only blinds (pot odds ~0.25), 0.30 > 0.25 so calling is fine (not folding). But 0.30 is below a raise threshold (~0.45), so just limp/call. If the opponent raised big, pot odds might become 0.4+, and since 0.30 < 0.4, the bot would fold to a large raise.

Post-Flop Discard Decision Strategy

The **discard round** is the defining feature of Toss or Hold'em. After seeing the 2-card flop, each player chooses one of their 3 hole cards to toss onto the board (making it public). The strategic goal is to **keep the best 2 cards** that maximize your final hand's equity while discarding the card that either: - Contributes the least to your hand's strength, **and** - (if you are discarding first) gives away as little useful information or advantage to the opponent as possible.

The baseline uses Monte Carlo simulation to guide this choice. Specifically, it calculates the **equity of the hand for each possible discard option**. For each of your 3 hole cards, imagine you discard that card and keep the other two – what is your win probability (equity) against a random opponent hand and random future cards? The bot's `stats.discard_equity()` function does exactly this: it simulates many random outcomes for each discard choice and returns the equity for each scenario ⁴⁹ ⁵⁰. For example, if your hole cards are [A♠, 7♥, 7♣] and the flop is [7♦, K♠], you have trips and likely you'd discard the irrelevant A♠ (keeping 7♥7♣). The simulation would confirm that discarding A♠ yields an extremely high win rate (since you keep trips) whereas discarding one of the 7s would yield far lower equity. The bot will pick the discard that gives the **highest equity** (with some randomness only if equities are very close) ⁵¹ ⁵².

Monte Carlo for Discard: Under the hood, for each candidate card to discard, the algorithm runs through random deals of: - The opponent's 2 hole cards (from the remaining deck), - The remaining future board cards (turn, river, and also the opponent's discard if they haven't discarded yet).

It then evaluates the outcome of the showdown: your best 5-card hand vs opponent's best 5-card hand, given that the discard card is now on the board ⁵³ ⁵⁴. It counts wins, losses, ties for each discard choice to estimate equity ⁵⁵ ⁵⁶. This simulation can be done by sampling a few thousand random combinations in a few milliseconds. In code, the baseline limits this to maybe 50–70 samples (or more if time allows) because an exact enumeration of all possibilities would be huge ⁵⁷ ⁵⁸.

The bot then **chooses the discard that maximizes win probability**. By doing this, it's effectively picking the card that is least useful to keep. In most cases this aligns with human intuition: **discard your weakest card**. For example: - If you have a pair among your 3 cards, you almost always keep the pair and discard the odd card out (the simulation will show keeping the pair yields higher equity). - If you have three cards of one suit (flush draw) or connected cards (straight draw), you might discard the one that doesn't fit the flush/straight. - If you have one high card and two low cards, likely discard one of the low cards (unless the flop strongly favors the low cards).

Position and Information Considerations: Toss or Hold'em adds a twist: the order of discards. The **out-of-position player discards first, and their discard is immediately revealed on the board** for the opponent to see ² ⁹. The in-position player then discards, also revealing that card. So both discards become known to everyone, but the second discarder has a crucial advantage of seeing an extra board card (the opponent's discard) before choosing their own. This can influence strategy in two ways:

- The first discarder (typically the player who was big blind) might want to be careful not only to pick their best hand, but also **not to "help" the opponent unnecessarily**. Since whatever they discard becomes a board card that both can use, discarding certain cards could immediately strengthen the opponent's hand. A dramatic example: in the official rules, player B (out of position) discarded the J♥ onto a flop of [7♥, 2♥], and it turned out player A was holding two hearts ⁹. By discarding the J♥, B put a third heart on board, **instantly giving A a flush** (A had 3♥4♥ in hand, and combined with the heart flop and the J♥ now on board, A had 5 hearts) ⁹. B not only lost their chance at a flush (they discarded their only heart), but also provided A a made flush. This example illustrates that discarding a card can have a **significant impact on the board texture**. If the flop has a potential flush or straight draw, adding a certain card could complete that draw for either player.

As the first discarder, you generally won't know exactly what the opponent holds, but you can avoid obvious blunders like **discarding a card that completes an open draw on the flop**. For instance, if the flop is two hearts and you have one heart in your hand, think twice about discarding that heart – putting a third heart on board greatly increases the chance someone (possibly your opponent) makes a flush. If that heart is not crucial to your own hand, it might seem safe to throw, but consider the downside: you might be "sharing" a strong draw with the opponent. The Monte Carlo simulation inherently accounts for some of this (it will show a reduced equity if discarding the heart often leads to the opponent winning more flush scenarios). Still, as a strategic rule: **don't give the opponent free outs**. Discard the card that helps you the least, *and* ideally doesn't complete a monster for the opponent. In practice, that means discarding low offsuit cards that don't improve the board much, and keeping (or at least not discarding) cards that could complete a straight/flush for the opponent.

- The second discarder (the dealer, in-position) has the luxury of seeing the board with the opponent's discard added. This can provide a read on what the opponent kept and also alter which of your cards is now weakest. The rules note that *"the dealer will see 3 cards on the board (including their opponent's discarded card) when deciding which card to discard"* ⁵. The opponent's discard might reveal something about their hand: for example, if they discarded an Ace, it suggests they might have two other very strong cards (perhaps a pair or suited connectors) that they deemed better than Ace-high. If they discarded a flush-suit card while the flop had a flush draw, that suggests they didn't prioritize the flush draw – possibly they have a made hand like a pair. These inferences are advanced, but the strategy can incorporate simple adjustments:

- If the opponent's discard now makes the board scary (e.g. completing a flush or high straight on board), evaluate if your hand needs to adjust. Sometimes, you might keep a card that “blocks” something the opponent could use. (Example: Suppose the board after their discard shows $4\spadesuit 5\spadesuit 6\spadesuit$ – a three-flush. If you hold one spade in your 3 cards, keeping it means at least you have a flush draw too or block one spade the opponent might need. Discarding that spade could put a fourth spade out or give up your only spade.) Generally, if their discard changed the board, re-run your Monte Carlo equity for each option (the bot's simulation naturally does this given the updated board) and choose accordingly.
- The baseline implementation actually applies a small “**information penalty**” if **your** discard will be visible (meaning you are discarding first) ⁵⁹. It slightly prefers discarding a lower-ranked card in those cases, to avoid revealing a high card. Mechanically, the code subtracts a factor proportional to the rank of each card from its equity score when you're the visible discarder ⁵⁹. This means if two discard choices are almost tied, it will choose to **toss the higher-ranking card if you are discarding second (hidden)**, but **toss the lower-ranking card if you are discarding first (visible)**, all else equal. The rationale is that revealing a high card (like showing the opponent you threw away a King or Ace) could give them more insight into what you kept. By instead discarding a lower card when you must show it, you conceal the true strength of your remaining hand a bit better ⁵⁹. This is a subtle strategy to **mitigate information asymmetry**.

In summary, the discard strategy is: **use Monte Carlo equity to choose the discard that maximizes your chances** after discarding, but incorporate the positional nuance. If you discard first, avoid extremely risky discards that massively improve the board for both (unless that card is absolutely useless to you and still yields the highest sim equity). If you discard second, take advantage of the additional info – e.g., if the opponent's discard seems to indicate a certain holding or has changed the board, factor that into which of your cards is now weakest. The bot's simulation does most of this heavy lifting by directly computing equities with the updated board ⁴⁹ ⁵⁰. The result of `discard_equity()` is a tuple of win probabilities for each of your 3 cards being discarded; the bot simply picks the index of the highest equity ⁵¹ (with a tiny random chance to deviate for unpredictability). This ensures the bot **keeps the best 2-card combination** for the situation nearly every time.

Finally, after choosing the discard, the strategy *records which card was discarded and whether it was visible* ⁶⁰. This can be useful later for opponent modeling (e.g., remembering what the opponent discarded and whether it was shown, to adjust their range).

Betting Strategy After the Discard (Post-Flop, Turn, River)

Once the discards are done, each player has 2 private cards remaining and there are 4 community cards on the board (flop + both discards). From here, the game resembles a somewhat unusual Texas Hold'em hand with 4 cards on board after the flop, and eventually 6 on board by showdown. The bot's betting strategy from this point onward revolves around **equity awareness, pot odds, and selective aggression**. The baseline approach is essentially an **equity-driven decision model**: at any point, use the estimated probability of having the best hand (win equity) versus the current pot odds to choose whether to bet, call, or fold, with some tweaks for raising and bluffing.

Equity Calculation: For any given street (after flop+discard, turn, or river), the bot uses `stats.estimate_equity()` to get its win probability if the hand goes to showdown ⁶¹. This function itself may do a Monte Carlo simulation similar to the discard calculation, but now it's simulating the

outcome of the entire hand (opponent's unknown cards and any remaining undealt board cards) without any further discards to consider. For example, on the turn (5 board cards visible), it will simulate random opponent hole cards and the one remaining river card to see how often our hand wins ²² ⁶². On the river (all 6 board cards known), equity estimation can be exact – simply compare our hand vs all possible opponent hands. The bot caches and seeds these simulations for speed and consistency ³⁴ ⁶³. The number of samples is adjusted by stage: e.g., fewer samples needed on the river or turn since there are fewer unknowns, more samples on flop because more uncertainty ⁶⁴ ⁶⁵. With this equity in hand (a number between 0 and 1), the strategy compares it to the **pot odds** of the situation.

Pot Odds and Thresholds: Pot odds represent the fraction of the total pot you must invest to call a bet. For instance, if the pot is 100 and your opponent bets 50, you must pay 50 to win 150 total, so your pot odds of winning are $50/150 = 0.33$ (33%). You should call only if your win probability is >33%, otherwise it's a losing proposition on average. The bot follows this principle: - If your equity (win probability) is less than the pot odds of calling, **fold** if folding is an option. This prevents calling when drawing slim. The baseline explicitly folds when `equity < pot_odds` in many cases ⁴⁸. For example, if you have only a 20% chance to win and the pot odds of calling a bet are 30%, folding is correct. - If your equity is greater than the pot odds, calling is profitable; thus the bot will at least **call** in that case if no better option. - If no bet is required (pot odds 0, e.g. you can check), the bot will gladly check and see the next card rather than fold ⁶⁶.

This baseline logic ensures the bot is making +EV (expected value) decisions in terms of chip equity for calls and folds. It essentially implements the classic **pot-odds-aware calling/folding** strategy ⁴⁸.

Value Betting and Raising: Merely calling when ahead is not enough to maximize winnings – we want to **bet or raise when we have an edge**. The strategy includes a mechanism to raise for value if our equity is comfortably above the breakeven point. Using a margin (buffer) by street, the bot sets a **raise threshold = pot odds + a margin** ⁴⁵. For example, on the flop the margin might be 15%, on the river 10% (since information is more certain) ⁴⁶. If our equity exceeds this threshold, it means we are a solid favorite and can benefit from putting more money into the pot. In that case, if a raise is available, the bot will raise. The baseline does this check in `_fallback_action`:

- First, it computes `pot_odds` for the current situation (using current pot and required call cost) ⁶⁷.
- It then computes a street-dependent `raise_margin` (e.g. 0.15 on flop, 0.12 on turn, etc.) ⁴⁶.
- If our equity > `pot_odds + raise_margin`, and we have the option to raise, it will initiate a raise of an appropriate size ⁴⁵. The code picks a raise amount in `_raise_size()` function, which likely chooses a moderate raise (not all-in unless warranted) based on our equity strength ⁴⁵. This might be half-pot or pot-sized depending on configuration, capped by some `raise_cap` so as not to go crazy huge ⁶⁸.

This means if we're well ahead, we press the opponent by betting for value – expecting to win more chips on average. For instance, if our equity is 70% and pot odds to call are 25%, definitely raise; even if pot odds after a small opponent bet are say 40%, $70\% > 40\% + \text{margin}$ (maybe ~55%), so still a raise. By requiring a margin above mere breakeven, we avoid raising on very marginal edges that could be due to simulation noise; we only value-raise when reasonably confident.

Bluffing Strategy: No good poker strategy is complete without occasional bluffs – especially heads-up, where you can't just wait for premium cards every time. The bot includes **bluff logic** that triggers under specific conditions: - It will consider a bluff raise when its equity is significantly *below* the pot odds (meaning

if it just plays straight up, it's likely to lose the hand) – i.e., when behind. Specifically, if `equity < pot_odds - 0.10` (10% worse than break-even) ⁶⁹, then a bluff might be in order **if** the situation is favorable. - A favorable bluff situation is determined by `_should_bluff(street, board)`. In the baseline, this returns False on early streets (no bluffing preflop or on the flop's first half) ⁷⁰, but allows consideration from turn or later. It also checks board texture: it avoids bluffing if the board is paired (since paired boards often means someone could have a full house or trips) or if the board is “flushy” (3 or more of one suit on board) ⁷¹ ⁷². Those board types tend to encourage calling (players often don't fold big hands or obvious draws), making bluffs less likely to succeed. If the board is unpaired and not three-to-a-flush, the bot will bluff only at a small fixed frequency (around 6%) ⁷³ – introducing a bit of randomness to avoid predictability. In practice, this means on a relatively dry board (no flush completed, not paired, e.g. $K\clubsuit-7\diamond-2\spadesuit-J\heartsuit$ after turn), if our equity is quite poor (say we have nothing), the bot might attempt a bluff raise 6% of the time, hoping the opponent folds. Bluff raises are sized using `_raise_size` with a bluff flag, usually selecting a smaller semi-bluff amount ⁷⁴. - The strategy will **never bluff into very dangerous boards** (like four of a kind or full house possibilities, where opponents never fold) ⁷⁵, and it keeps the frequency low so that it doesn't spew chips frequently. This aligns with the idea of “volatility maximization without donating EV” – i.e., add some bluffing to increase variance and exploit opponents who fold too often, but do it in spots that have a decent chance to work and not too often to be exploited.

Putting it all together: After the discard, the betting decisions at each street follow a flow: 1. **Compute equity** vs opponent using Monte Carlo (if flop/discard or turn) or exact (if river). 2. **Calculate pot odds** of any call/check decision. 3. **If a fold is available and equity is well below required (even considering a tiny margin), fold.** This protects us from chasing with weak draws or second-best hands when odds aren't offering enough. 4. **If we have a strong edge (equity well above threshold), and raising is possible, bet/raise for value.** This applies both when we're the first to act (bet out if strong) or responding to an opponent's bet (put in a raise). 5. **If we're very weak but see an opportunity (and conditions met), bluff occasionally.** For example, if opponent bets and our hand is trash but board is not obviously in their favor, we might check-raise bluff at a low frequency. If we're first to act on river with a busted draw, a small bluff bet could steal the pot if the opponent has weak showdown value. 6. **Otherwise, call or check.** If none of the above conditions trigger, the default is to continue in the hand without raising. So if equity is around the breakeven or slightly above it but not enough for a confident raise, the bot will usually just call. If it's our turn and we don't have a great reason to bet (e.g., medium-strength hand that prefers pot control), the bot can check and see what the opponent does, or call a moderate bet.

The baseline indeed implements such logic. For instance, after computing equity and pot odds, there's a check: **if raise is available:** - If `equity > (pot_odds + margin)` and a minimum raise is feasible, do a `RaiseAction` (value raise) ⁴⁵. - Else if equity is much lower than pot odds (by >0.1) and the bluff conditions `_should_bluff` are true, it will attempt a bluff `RaiseAction` ⁷⁶. If none of that, then: - If equity is below pot odds by more than a small `call_margin`, then **Fold** ⁷⁷. - If we can check for free (no cost), do that ⁷⁸. - If we can call (and haven't folded), then **Call** ⁷⁸. - Otherwise, if nothing else fits, fold as a last resort (this rarely happens except maybe in some edge all-in scenarios) ⁷⁹.

These rules ensure the bot is generally **playing solid poker**: - Folding when likely beat or drawing dead relative to price, - Calling with decent hands or pot odds, - Raising with strong advantages, - And bluffing just enough to keep opponents guessing.

Adjustment for 6 board cards: One thing to note: with 6 community cards by the end, the average hand strength is higher than in normal Hold'em (more cards means more chances to make strong hands). This

means some marginal draws or one-pair hands might be less valuable because often the best hand by river could be two-pair, trips or better due to the extra card. The bot's Monte Carlo equity calc inherently accounts for this (it will likely show lower equity for one-pair on a 6-card board than one might expect in 5-card board poker, because the opponent has more outs to improve). Strategically, this suggests playing a bit more cautiously with medium-strength hands – e.g., a single pair on the flop of a 4-card board might not hold up by river as often. Our strategy thresholds (raise_margin, call_margin) can be tuned to reflect that. The baseline margins do shrink on later streets (from 15% on flop to 10% on river for raises) ⁴⁶, which is logical because by the river equity estimates are more certain and differences are smaller. It also means by the river, if we have a marginal hand and the opponent bets big, folding is often correct unless equity is clearly there.

Opponent Modeling and Strategy Adaptation

While the above strategy is largely **static** (i.e., it doesn't directly adjust to a specific opponent's tendencies except via the basic bluff conditions), incorporating opponent modeling can significantly enhance performance in a competition setting. The baseline code provides a framework for tracking opponent actions ⁸⁰ and even placeholders for updating opponent hand range estimates ⁸¹ ⁸², though these were not yet implemented. To win the competition, we should use any predictable patterns in opponents to our advantage: - **Tracking Frequencies:** The bot should keep counts of how often the opponent folds, calls, raises, and showdowns ⁸³ ⁸⁰. Over many hands, if we notice (for example) that the opponent folds to most bets (high fold_count vs call_count), we can increase our bluff rate and value-bet thinner (bet more often because they are likely to fold anything mediocre). Conversely, if an opponent calls very frequently (low fold_count, high call_count), we should bluff rarely and only bet big with strong hands (a "calling station" demands a value-heavy approach). - **Range inference from discards:** The discard action provides a unique information point. Seeing **what card an opponent discarded** face-up can narrow their possible holdings. For instance, if an opponent discards a very high card (say Q♣) on a flop where that card had no immediate synergy, it might mean their other two cards formed a strong pair or draw, making the queen extraneous. If they discard a card that *would* have given them a flush draw, it implies they either already have a better draw or made hand, or they simply don't value draws – which is useful to know. We could implement a heuristic: *update opponent range after seeing discard*. For example, if a player discards a heart when the flop is two hearts, we infer that player likely doesn't have two hearts (otherwise discarding one is odd) and maybe they held only one heart which they gave up, or they prioritized something else. The baseline suggests a function `update_range_after_discard(range3, discarded_card)` for this purpose ⁸⁴. Though not implemented, an idea would be to remove all 3-card combinations from the opponent's pre-discard range that wouldn't discard that particular card. This is complex, but even a simple rule like "if they discarded a high card, assume their kept cards have some synergy (pair or suited)" or "if they discarded a middle card and kept an Ace, maybe they had Ace-x suited" can inform how we play the turn/river against them. - **Adjusting Bluffing and Calling:** If an opponent is very aggressive (high raise_count relative to showdown_count), they might be bluffing often. We can loosen up our calling threshold (call more often with moderate hands) to catch their bluffs. If an opponent is very passive (almost never raises, mostly calls or folds), we should bluff them more and also fold more when they suddenly bet/raise (because a passive player's aggression usually means a real hand). - **Discard Tendencies:** Some opponents might always discard certain types of cards (e.g., always discard the lowest card regardless of board). If we spot that, we could exploit it. For example, if an opponent *always* keeps suited cards (never breaks a potential flush), we can anticipate that in their strategy and perhaps bluff more on flush scare boards when they *don't* discard a suit (meaning they likely have it). These are very opponent-specific, but tracking a few rounds can reveal patterns.

Because there are **1000 rounds**, there is ample opportunity to gather statistics. The strategy can incorporate a simple learning mechanism: each round update an opponent model (the baseline has `update_opponent_model()` for each observed action ⁸⁰ and placeholders for learning functions ⁸⁵). Over time, tune parameters like our `bluff_rate` or calling margins dynamically in response. For instance: - If opponent folds a lot, increase our `_bluff_rate` from the default 3% towards, say, 10%, and decrease our raise thresholds (we can value bet thinner). - If opponent never folds and calls down with weak holdings, set `_bluff_rate` to 0% (don't bluff them at all) and only bet with strong hands, otherwise just check/fold marginal hands. - If opponent's discard choices seem poor (e.g., they consistently discard in a way that lowers their equity – perhaps identifiable if we see them reveal strange discards), we can exploit by calling more often because they might be overestimating their hand or underestimating draws.

The baseline's plan (Phase 3: Opponent modeling ⁸⁶) includes tracking fold frequency by street, raise/call ratios, etc., which aligns well with these adaptations. Implementing even a simple version of this can give a competitive edge.

Performance and Implementation Considerations

Winning the competition not only requires good strategy but also **efficient implementation**. The bot must operate within time limits (both per decision and overall per match) without timing out. Here are key performance strategies, many of which the baseline already uses or plans:

- **Use C++ for Heavy Lifting:** As mentioned, leveraging the compiled PokerStove evaluator is crucial for speed. By building the PokerStove library into the project (as a Python module) ⁸⁷, the bot can evaluate hand ranks extremely fast. Every Monte Carlo simulation uses thousands of hand evaluations internally, so using PokerStove's C++ code for that will drastically speed up simulations (on the order of 5-10x faster than pure Python) ¹⁹. Make sure the environment is configured to use this (e.g., setting `NEUROPOKER_EVAL_BACKEND=pokerstove` so that `_USE_POKERSTOVE` is true in code ⁸⁸).
- **Parallel Sampling:** The code hints at multi-processing support for Monte Carlo (`_DEFAULT_MC_WORKERS = 2`) ⁸⁹ ⁹⁰. If the framework allows, using 2 workers to split the simulations can roughly double the number of samples in the same wall-clock time. For example, one process simulates half the trials, another does the other half, then combine results. This should be used when the number of samples is large enough to justify the overhead (the baseline sets `_DEFAULT_MC_PARALLEL_MIN = 200` as a threshold above which to use parallelism) ⁸⁹ ⁹⁰. In practice, for ~100 samples (the default in many spots), the overhead might not be worth it, but for deeper calculations it could be. Since speed is a priority, we should exploit parallelism especially in critical junctures (like big pots or final hand calculations) to get more accuracy without slowing down.
- **Sampling Budgets:** The strategy already defines different Monte Carlo sample counts per street ⁶⁴ ⁶⁵ – e.g., fewer samples early to act faster, more samples later when decisions are crucial. It also reduces samples if the game clock is running low ⁹¹. Tuning these parameters is important. We want enough samples to get a reliable equity estimate, but not so many that we waste time. For instance, using 120 samples on the river (where only opponent's cards are unknown) might be overkill when exact calculation is possible by iterating through all remaining card combinations (which is only $C(45,2) \approx 990$ possible opponent holdings if no unknown board cards). We might

implement a special case: on the river, do a full enumeration of opponent hands for exact equity – this is feasible and ensures no simulation error. On the flop (with turn and river unknown, and opponent hand unknown), full enumeration is impossible (combinatorial explosion), so Monte Carlo is fine. The default ~80 samples on flop may be a bit low; if time allows, increasing to a few hundred could improve decision quality. One could even use an **adaptive sampling**: sample until the confidence interval of the equity estimate is below a threshold or until time is up. But given the time constraints per move are likely small (maybe ~50ms), fixed small sample counts with caching is a safer approach.

- **Caching & Determinism:** The bot caches expensive computations like discard equities and general equities for particular states ⁹² ²⁷. For example, if the same flop and hand situation arises again (not unlikely in 1000 hands), it can reuse the result instead of re-simulating. It also seeds random draws based on the state hash to ensure the same situation yields the same equity estimate ³⁴. This determinism helps debugging and prevents oscillating decisions (which could confuse learning). We should ensure the cache sizes (like 200k entries for equity cache) ⁹² are sufficient but not memory-exhaustive. Generally, caching shows diminishing returns in poker since each situation is somewhat unique, but certain sub-computations (like evaluating the same 5-card hand rank) happen *a lot*, hence the evaluator cache of 400k entries ²⁷ is very useful. The strategy should continue to take advantage of caching at every level possible.
- **Avoid Redundant Simulation:** We should be careful not to simulate equity when not needed. For example, if a decision is obvious (equity = 0 or 1 in some extreme case, or pot odds = 0 requiring no call), skip it. If we have the nuts (100% win), we don't need simulation to decide to raise. If we have 0% (impossible scenario aside from folding), likewise. Another optimization: pre-calculate **preflop equity tables** for 3-card vs 3-card matchups offline. It might be possible to compute exact or near-exact head-to-head equities for all 3-card combinations vs random opponent using a script and include that data. Then the bot could simply look up preflop equity instead of simulating or using a heuristic. However, since the heuristic is already in place and reasonably accurate, this is a secondary improvement.
- **Time Management:** The competition likely has a total time bank. The bot already reduces simulation lengths if the game clock is low (e.g., if less than 20 seconds remain, it halves the samples) ⁹¹. It's wise to incorporate a gradual reduction in thinking time as the match goes on, or if we detect that decisions are taking close to the limit. Ensuring no timeout is paramount; a single timeout might forfeit a round. So our strategy might err on the side of slightly fewer samples and faster replies, especially in trivial spots, to save time for more complex decisions.

By adhering to these performance strategies, we ensure that our comprehensive strategy doesn't fall apart due to slow computations. **Fast, reproducible Monte Carlo evaluations** guided by robust strategy logic will allow the bot to play 1000 rounds without lag. In testing, we should validate that typical decisions (folds, calls) compute near-instantly (under a few milliseconds) thanks to caches and low sample counts, and even tough decisions (big pot river calls, etc.) complete within maybe tens of milliseconds. The provided baseline numbers (e.g., default 120 samples, 0.03s max on later streets) ³⁰ ⁶⁵ indicate the bot is already aiming to keep each decision under 30ms or so, which is a good target.

Conclusion

To win the Toss or Hold'em competition, our bot's strategy must intelligently leverage its tools (fast equity calculators and simulations) and adapt to the nuances of this unique variant – all while operating under strict time constraints. We have outlined a comprehensive strategy that:

- Uses **Monte Carlo simulations** and a PokerStove evaluator to accurately assess hand strength and win probabilities in real time, guiding decisions at each phase ²⁶ ²² .
- Maximizes speed via caching, optimized sample sizes, and compiled code, ensuring fast hand evaluations (with PokerStove back-end and Deuces as fallback) for the 8-card showdown scenario ²⁷ ¹⁹ .
- Plays a solid **preflop game** with 3 hole cards, employing a heuristic to rank starting hands ³⁹ and making +EV choices: raising strong tri-card holdings, folding the weakest, and otherwise seeing flops for cheap when appropriate.
- Executes an equity-driven **discard strategy**, keeping the best possible 2-card combination. It simulates each potential discard's outcome and selects the card whose removal yields the highest winning chances ⁴⁹ ⁵⁰ . It also accounts for the information reveal: when discarding face-up, avoid obvious giveaways or aiding the opponent ⁵⁹ . As the second discarder, use the advantage of seeing the opponent's discard to refine your choice (as in the example where discarding a heart completed a flush for the opponent – our bot will recognize if a discard choice significantly drops its equity due to such effects) ⁹ .
- Follows through with robust **betting strategy** post-flop: folding when the odds are against us ⁴⁸ , calling or checking when appropriate, and raising for value when we're ahead by a safe margin ⁴⁵ . We also incorporate **strategic bluffs** on boards and situations where they are likely to succeed, at a low frequency to remain unpredictable ⁷¹ . The bot's decisions continuously reference the equity vs. pot odds comparisons, essentially performing an automatic EV calculation for each action – a fundamental part of poker strategy ⁴⁸ .
- Can **adapt to opponents** over the long run by observing their folding frequency, aggression, and discard habits. This allows exploitation: for example, against a timid opponent our bot will bluff more and steal pots, whereas against a calling-station it will tighten up on bluffs and focus on value betting. Though the baseline provides the hooks for opponent modeling ⁸⁰ ⁸¹ , implementing even simple adjustments can increase our win rate as the match progresses.

Every element of this strategy is geared toward **maximizing expected value while minimizing computational delay**. By keeping decisions fast and grounded in sound poker math, the bot can play 1000 rounds of Toss or Hold'em efficiently and effectively. The Monte Carlo approach gives it a dynamic, situation-specific strategy rather than a static rule-based one, which is important in the unpredictable arena of poker. Coupled with the discard mechanism mastery and opponent adaptation, our bot should be well-equipped to outplay opponents and claim victory in the competition.

¹ ² ³ ⁴ ⁵ ⁶ ⁷ ⁸ ⁹ ¹⁰ ¹¹ ¹² ¹³ ¹⁴ ¹⁵ variant.pdf

file:///file_000000001e0071f4ba634f631a20d2a7

¹⁶ ⁸⁶ plan.md

<https://github.com/nyx-haile/pokerbots/blob/cf3df3954a386f6e3b6d8570cb083a1344e132d8/plan.md>

17 18 20 21 22 23 24 27 28 29 30 31 32 33 34 35 37 38 39 40 41 42 43 53 54 55 56 62 63 88

89 90 92 **stats.py**

<https://github.com/nyx-haile/pokerbots/blob/cf3df3954a386fbe3b6d8570cb083a1344e132d8/neuropoker/stats.py>

19 **deuces-README.md**

<https://github.com/nyx-haile/pokerbots/blob/cf3df3954a386fbe3b6d8570cb083a1344e132d8/documentation/deuces-README.md>

25 **cs.columbia.edu**

<https://www.cs.columbia.edu/~sedwards/classes/2023/4995-fall/proposals/PokerEquity.pdf>

26 **Watson Chua's Poker Win Calculator: An AI Engineer's Deep Dive into the MCP Server**

<https://skywork.ai/skypage/en/poker-win-calculator-ai-engineer/1981606044208566272>

36 44 45 46 47 48 49 50 51 52 57 58 59 60 61 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78

79 80 81 82 83 84 85 91 **strategy.py**

<https://github.com/nyx-haile/pokerbots/blob/cf3df3954a386fbe3b6d8570cb083a1344e132d8/neuropoker/strategy.py>

87 **AGENTS.md**

<https://github.com/nyx-haile/pokerbots/blob/cf3df3954a386fbe3b6d8570cb083a1344e132d8/AGENTS.md>